Rapport POO2

Tarash Pavlo
Veverita Ion

Mai 2025

1 Diagram UML

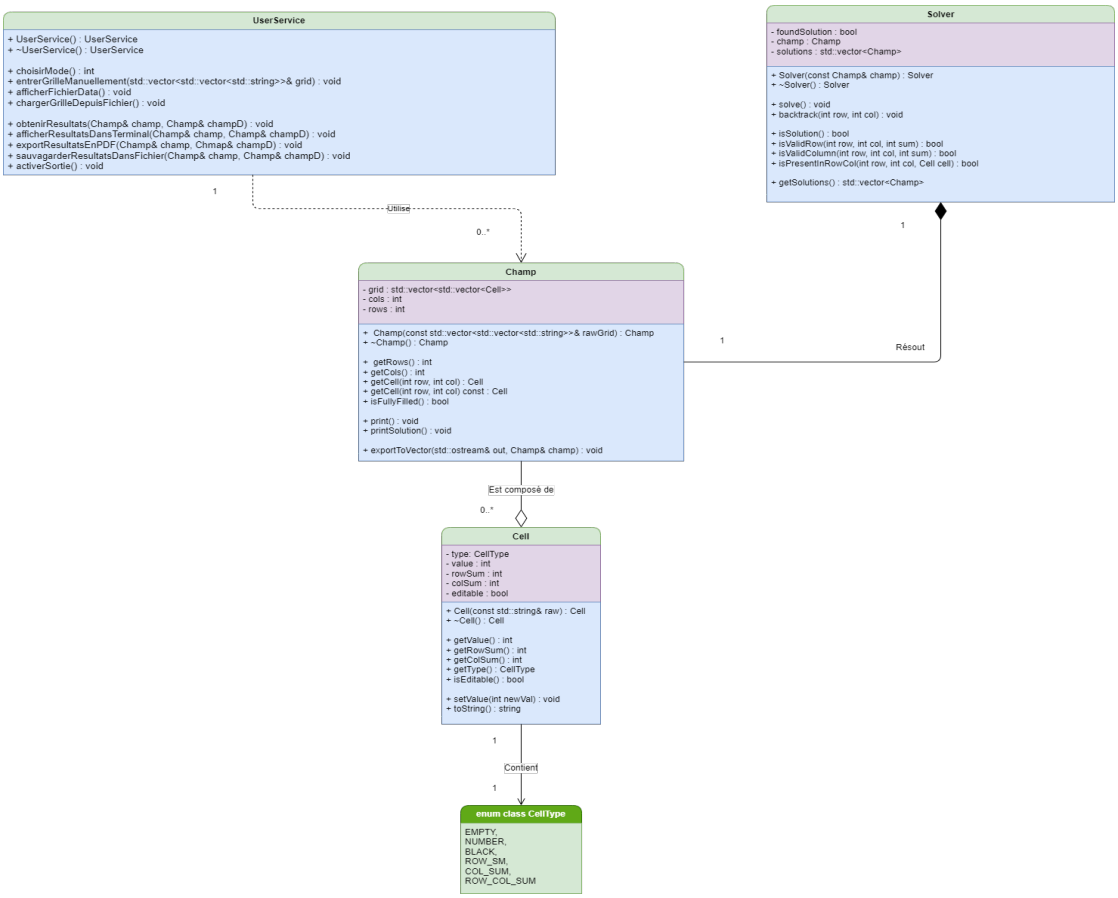

Figure 1 – UML

Compilation

Pour compiler le programme, exécutez la commande :

```
make
```

Pour accorder les droits d'exécution au script `vector2tex.sh`, utilisez :

```
make chmod
```

Pour installer LaTeX (si ce n'est pas déjà fait, cette étape peut être ignorée si inutile) :

```
make latex
```

Pour nettoyer les fichiers générés :

```
make clean
```

Exécution

Pour exécuter le programme, utilisez la commande suivante après la compilation :

```
./bin/main
```

Ensuite, suivez simplement les instructions affichées dans le terminal.

Attention :
Dans la grille, une case contenant un nombre est considérée comme une valeur fixe (ou donnée initiale ou constante). Cette valeur **ne sera pas modifiée** lors du processus de résolution. Il faut faire attention aux valeurs, car elles définissent les contraintes fixes de la grille.

2 Instructions d'Utilisation du Programme

```
#####################################################################
#-------Bienvenue dans le programme de solveur de jeu Kakuro------#
#####################################################################
Veuillez choisir un mode de saisie de la grille :
0. Quitter
1. Saisir manuellement
2. Importer depuis un fichier
>>
```

FIGURE 2 – Debut de programme

Choix du mode d'entrée

- **0** : Quitter le programme.
- **1** : Saisie manuelle de la grille.
- **2** : Importation de la grille depuis un fichier dans le répertoire `/data/`.

Saisie manuelle

Vous devrez entrer :

- le nombre de lignes ;
- le nombre de colonnes ;
- les valeurs de chaque cellule selon les instructions.

```
Veuillez entrer chaque cellule (ligne par ligne) :
Attention : Assurez-vous que la grille saisie est **valide**.
Sinon, le solveur pourrait ne pas trouver de solution ou se comporter de manière inattendue.

Format:
  _   => case vide (editable)
  #   => case noire (non-editable)
  12\ => 12 est la somme de colonne uniquement
  \7  => 7 est somme de ligne uniquement
  5\3 => 3 est somme de ligne et 5 est la somme de colonne
---------------------------
Ligne 1 (séparer par espace):
```

FIGURE 3 – Saisie manuelle(1)

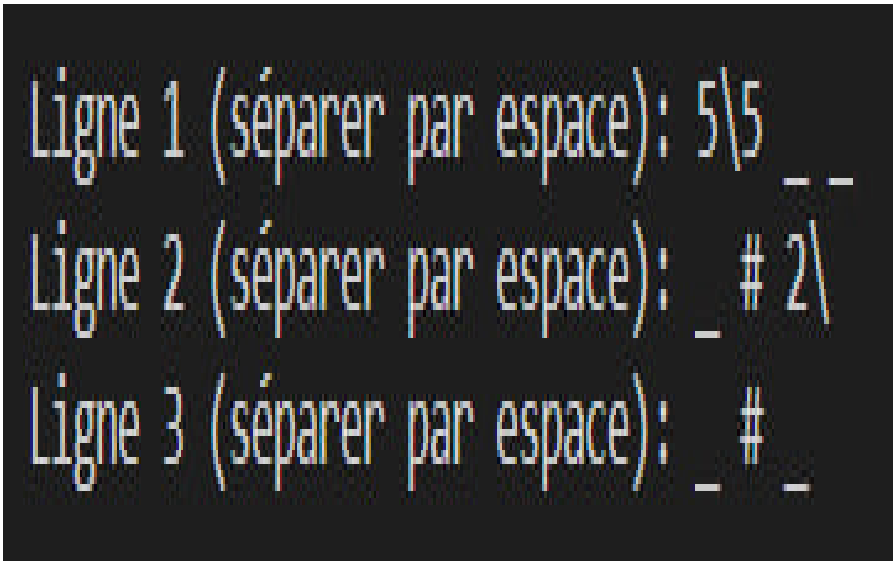

FIGURE 4 – Saisie manuelle(2)

Exemple de grille

```
Depart :
5\5 _ _
_   # 2\
_   # _
Solution :
5\5 1 4
1   # 2\
4   # 2
```

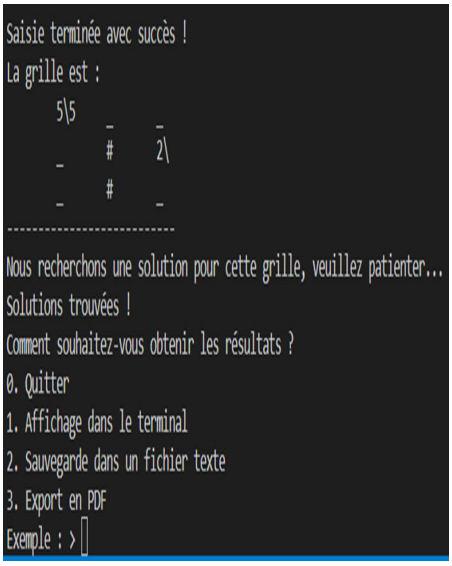

FIGURE 5 – Obtenir le resultat : 3

Résultat

Si la grille est correcte et une solution est trouvée, vous aurez le choix entre :

- **0** : Quitter.
- **1** : Afficher dans le terminal.
- **2** : Sauvegarder dans un fichier texte (nom unique).
- **3** : Exporter en PDF (inclut grille de départ et solution).

Voici l'image représentant la grille de depart et la solution :

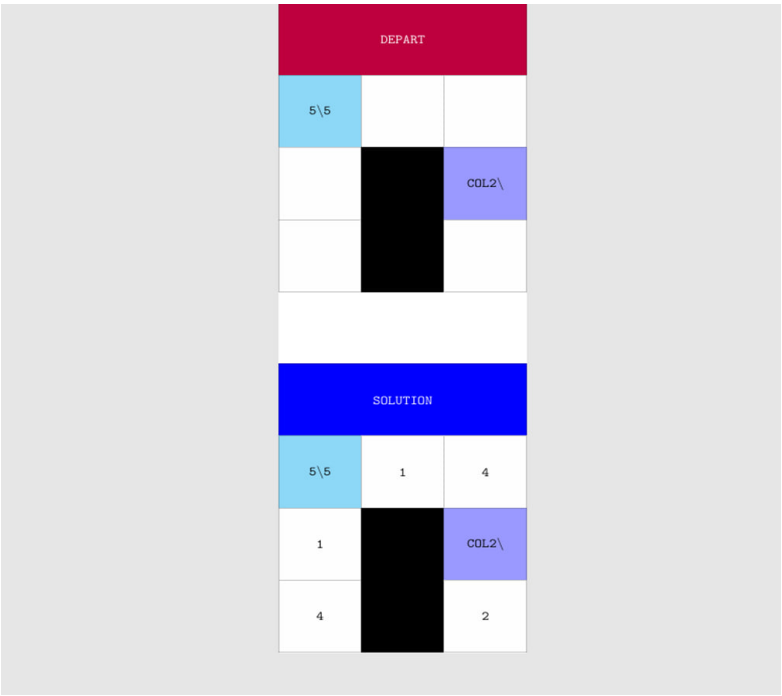

FIGURE 6 – Exemple de grille(pdf)

Introduction

Ce document présente la structure et le fonctionnement d'un programme écrit en langage C++. Il s'agit d'un solveur pour le jeu de Kakuro, basé sur une grille, qui utilise des mécanismes de résolution automatique par backtracking.

3 Description des Classes

3.1 Enum Class `CellType`

Un type énuméré sécurisé représentant les différents types de cellules :

— **EMPTY** : case vide.

— **NUMBER** : case contenant un nombre initial bien positionné.

— **BLACK** : case noire.

— **ROW_SUM** : case contenant la somme d'une rangée (notation : `\x`).

— **COL_SUM** : case contenant la somme d'une colonne (notation : `x\`).

— **ROW_COL_SUM** : case contenant la somme d'une rangée et d'une colonne (notation : `x\y`).

3.2 Classe `Cell`

Attributs privés

- `CellType type` : le type de la cellule. Chaque cellule ne peut posséder qu'un seul type à la fois, par exemple `BLACK`, et aucun autre.
- `int value` : valeur de la cellule.
- `int rowSum` : somme de la rangée (valide uniquement pour `ROW_SUM` et `ROW_COL_SUM`).
- `int colSum` : somme de la colonne (valide uniquement pour `COL_SUM` et `ROW_COL_SUM`).
- `bool editable` : indique si la cellule est modifiable (seules les cellules de type `EMPTY` le sont).

Méthodes publiques

- Constructeur : `Cell(const std::string& raw);`
- Destructeur : `~Cell();`
- Getters :
  - `int getValue() const;`
  - `int getRowSum() const;`
  - `int getColSum() const;`
  - `CellType getType() const;`
  - `bool isEditable() const;`
- Setter : `void setValue(int newVal);`
- Méthode d'affichage : `std::string toString() const;`

3.3 Classe `Champ`

Attributs privés

- `std::vector<std::vector<Cell» grid` : une grille de cellules représentant l'ensemble du plateau de jeu.
- `int rows` : le nombre de rangées dans la grille.
- `int cols` : le nombre de colonnes dans la grille.

Méthodes publiques

- Constructeur : `Champ(const std::vector<std::vector<std::string>>& rawGrid);`
- Destructeur : `~Champ();`
- Getters :
  - `int getRows() const;`
  - `int getCols() const;`
  - `const Cell& getCell(int row, int col) const;`
  - `Cell& getCell(int row, int col);`
- Vérification : `bool isFullyFilled() const;`
- Affichage :
  - `void print() const;` -> affiche la grille initiale, en remplaçant les cellules de type `EMPTY` par des _ (soulignés), ce qui signifie que ces cases sont encore vides.
  - `void printSolution() const;` -> affiche la grille complète, incluant les valeurs numériques trouvées pour les cellules de type `EMPTY` après résolution.
- Export : `void exportToVector(std::ostream& out, Champ& champ) const;` -> Écris dans un fichier les coordonnées de chaque case afin que le script puisse les traduire en LaTeX.

3.4 Classe `Solver`

Attributs privés

- `bool foundSolution = false;` -> Indique si une solution a été trouvée.
- `Champ champ;` -> La grille principale sur laquelle la recherche de solution est effectuée.
- `std::vector<Champ> solutions;` -> Vecteur contenant l'ensemble des solutions trouvées pour la grille.

Méthodes publiques

- Constructeur : `Solver(const Champ& champ);`
- Destructeur : `~Solver();`
- `void solve();` -> Lance le methode backtrack(), avec 0 0
- `std::vector<Champ> getSolutions() const;` -> retourne le solutions

Méthodes privées

- `bool isValidRow(int row, int col, int sum);`
  Vérifie si **la ligne** de la grille est valide (par rapport à la somme attendue et aux valeurs présentes).
- `bool isValidColumn(int row, int col, int sum);`
  Vérifie si **la colonne** de la grille est valide.
- `bool isSolution();`
  Vérifie si la grille représente une solution valide. Cela inclut :
  1. La grille est complètement remplie.
  2. Toutes les lignes et colonnes respectent les contraintes de somme.
  3. Il n'y a pas de doublons dans les lignes et les colonnes.
- `bool isFullyFilled();`
  Vérifie si **toutes les cellules jouables** de la grille sont remplies.
- `bool isPresentInRowCol(int row, int col, Cell cell);`
  Vérifie s'il n'y a pas de **doublon** (même valeur) dans la ligne ou la colonne pour la cellule donnée.
- `void backtrack(int row, int col);`
  Fonction principale de **backtracking** qui teste toutes les combinaisons possibles pour remplir la grille correctement.

3.5 Classe `UserService`

Méthodes publiques

- `UserService();`
  Constructeur de la classe.
- `~UserService();`
  Destructeur de la classe.
- `void activerSortie();`
  Active la sortie principale du programme.
- `int choisirMode();`
  Demande à l'utilisateur de choisir le mode de chargement de la grille.
- `void afficherFichierData();`
  Affiche les fichiers présents dans le répertoire `/data/`.
- `void entrerGrilleManuellement(std::vector<std::vector<std::string>>& grid);`
  Affiche les règles de saisie pour la grille, puis permet à l'utilisateur de la remplir manuellement.

- `void chargerGrilleDepuisFichier(const std::string& filename, std::vector<std::`
  Charge une grille depuis un fichier texte.
- `void obtenirResultats(Champ& champ, Champ& champD);`
  Demande à l'utilisateur de choisir le mode de génération des résultats (affichage, fichier, ou PDF).
- `void afficherResultatsDansTerminal(Champ& champ, Champ& champD);`
  Affiche les résultats directement dans le terminal.
- `void sauvegarderResultatsDansFichier(Champ& champ, Champ& champD);`
  Sauvegarde les résultats dans un fichier texte dans le répertoire `resultats/`.
- `void exporterResultatsEnPDF(Champ& champ, Champ& champD);`
  Exporte les résultats au format PDF dans le répertoire `resultats/`.

L'algorithme principal consiste en plusieurs étapes, avec les difficultés suivantes :

1. Difficulté liée à l'incrémentation des cases

La première difficulté est de ne pas parcourir toutes les cases dans le sens de l'incrémentation des chiffres, afin de ne pas augmenter la complexité, qui est déjà assez grande. En effet, certaines cases ne servent qu'à indiquer qu'il s'agit d'une case noire ou à donner la somme des colonnes ou des rangées. Ainsi, dans la fonction `backtrack()`, on ne considère que les cases `EMPTY` pour l'incrémentation afin de trouver une solution.

Pour cela, on utilise une énumération pour chaque type de case et un booléen qui définit si on peut modifier la cellule ou non.

2. Vérification de la validité de la solution

La deuxième difficulté réside dans la vérification de la validité de la solution. Pour ce faire, plusieurs fonctions sont utilisées :

- `isValidRow()` : Vérifie la validité de la ligne.
- `isValidColumn()` : Vérifie la validité de la colonne.
- `isFullyFilled()` : Vérifie si la grille est complètement remplie.
- `isPresentRowCol()` : Vérifie la présence d'un élément dans la ligne et la colonne.

La fonction `isFullyFilled()` est utilisée pour vérifier si la grille est entièrement remplie, car il est inutile de vérifier une grille vide. Les autres fonctions vérifient la validité des sommes ainsi que la présence des éléments dans les lignes et les colonnes.

3. Exportation de la grille dans un fichier PDF

La troisième difficulté concerne l'exportation de la grille de départ et de la solution dans un fichier PDF. Une fonction permet d'écrire la grille dans un fichier `output.vector`, qui est ensuite transformé en LaTeX puis en PDF.

Les étapes de l'algorithme

Les étapes de l'algorithme sont les suivantes :

1. Mode d'entrée.
2. Chargement de la grille.
3. Recherche de la solution.
   L'algorithme incrémente chaque case vide à 1, et à chaque étape, dès que la grille est complètement remplie, il appelle la fonction `isSolution()` pour vérifier si la solution est valide ou non. La fonction `isSolution()` utilise à son tour les fonctions `isFullyFilled()`, `isValidRow()` et `isValidColumn()`.

   Avec l'algorithme de backtracking, on combine toutes les valeurs possibles pour chaque case afin de trouver la solution. Cet algorithme explore différentes configurations de manière récursive, et revient en arrière (backtrack) lorsque nécessaire pour tester d'autres possibilités.
4. Mode de sortie : écriture dans un fichier ou génération du PDF ou affiche dans le terminal.

Le projet a été réalisé en binôme. Le travail a été effectué de manière équitable car il est difficile de dire que l'un a fait plus que l'autre, car chacun de nous a contribué avec sa part de code, d'idées, de réalisations et bien d'autres aspects pour mener à bien ce projet.